

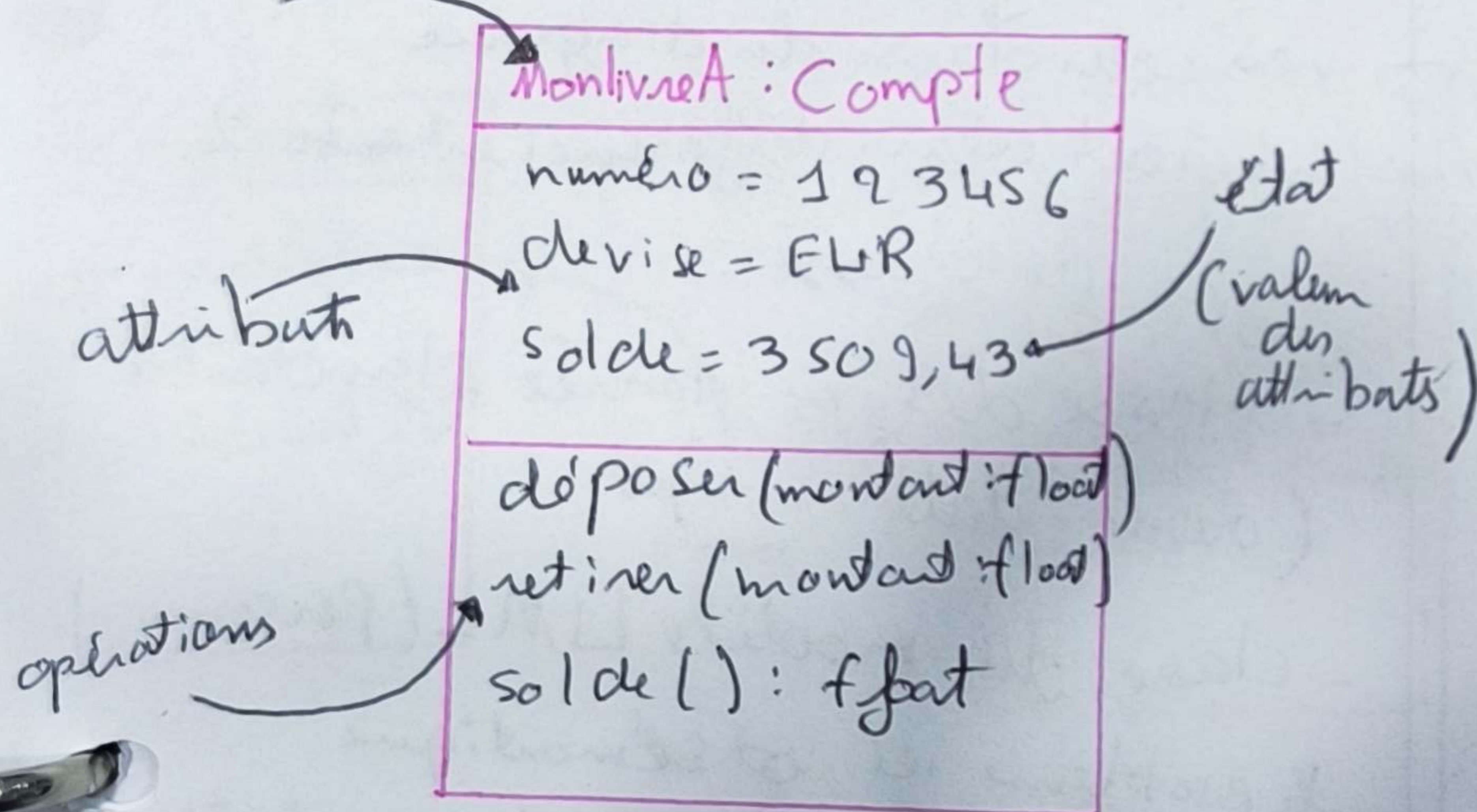
# Diagrammes de classes

## Objet et classe

### Objet :

- Entité concrète ou abstraite du domaine d'application
- Décrit par :
  - identité (adresse mémoire)
  - état (attributs)
  - comportement (opérations)

Soit l'instance suivante (l'instance) :



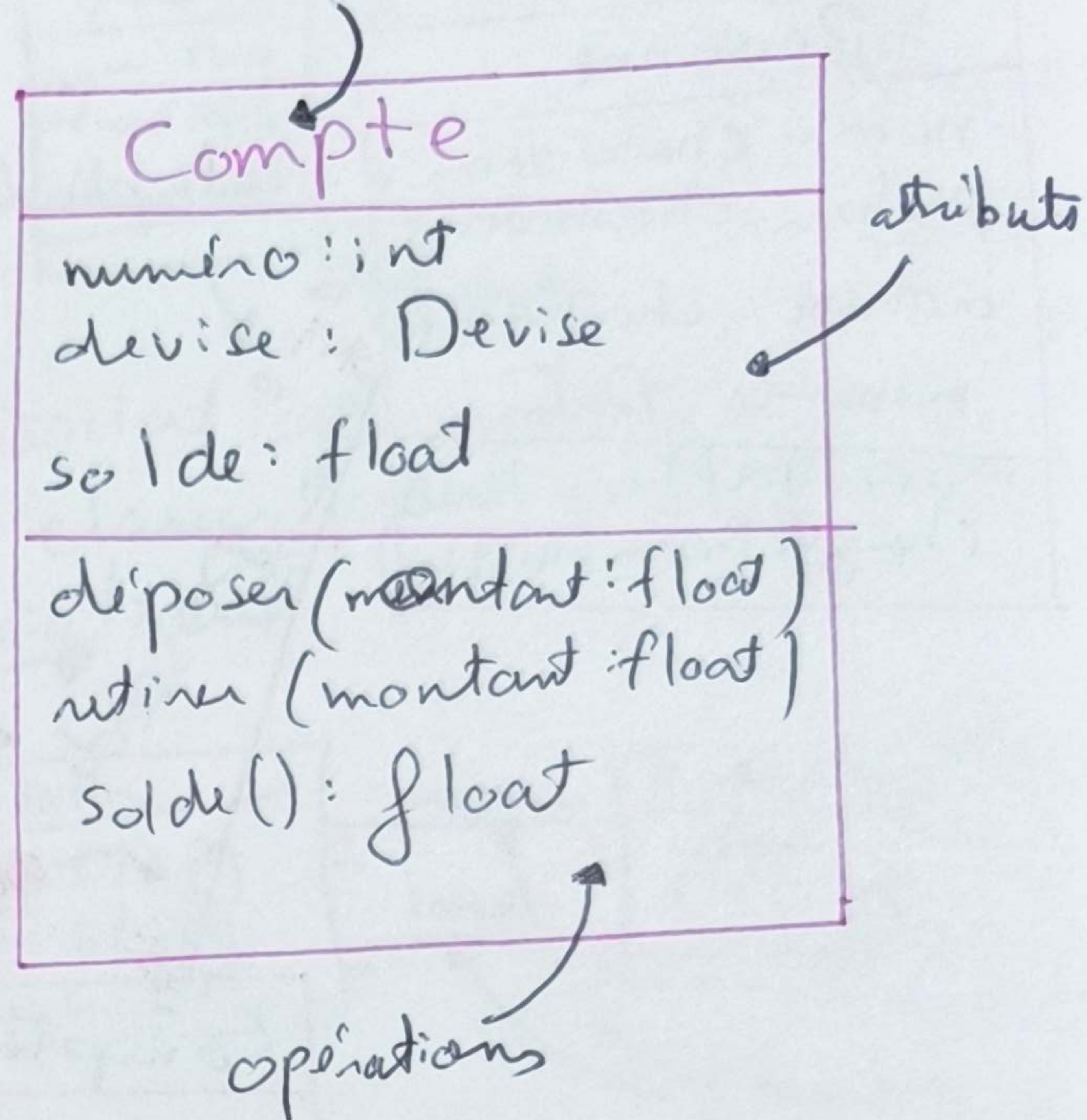
### Classe :

une classe est un Regroupement d'objets de même nature (mêmes attributs + mêmes opérations)

Objet = instance d'une classe

selon l'exemple précédent l'instance Monlivre A est une instance de la classe Compte suivante :

nom de la classe



### Attributs :

- Caractéristique partagée par tous les objets de la classe
- Associe à chaque objet une valeur (cas d'attribut d'objet pas attribut de classe)
- deux objets peuvent avoir le même état d'attributs mais ils n'auront jamais la même identité & adresses mémoire

### Opération :

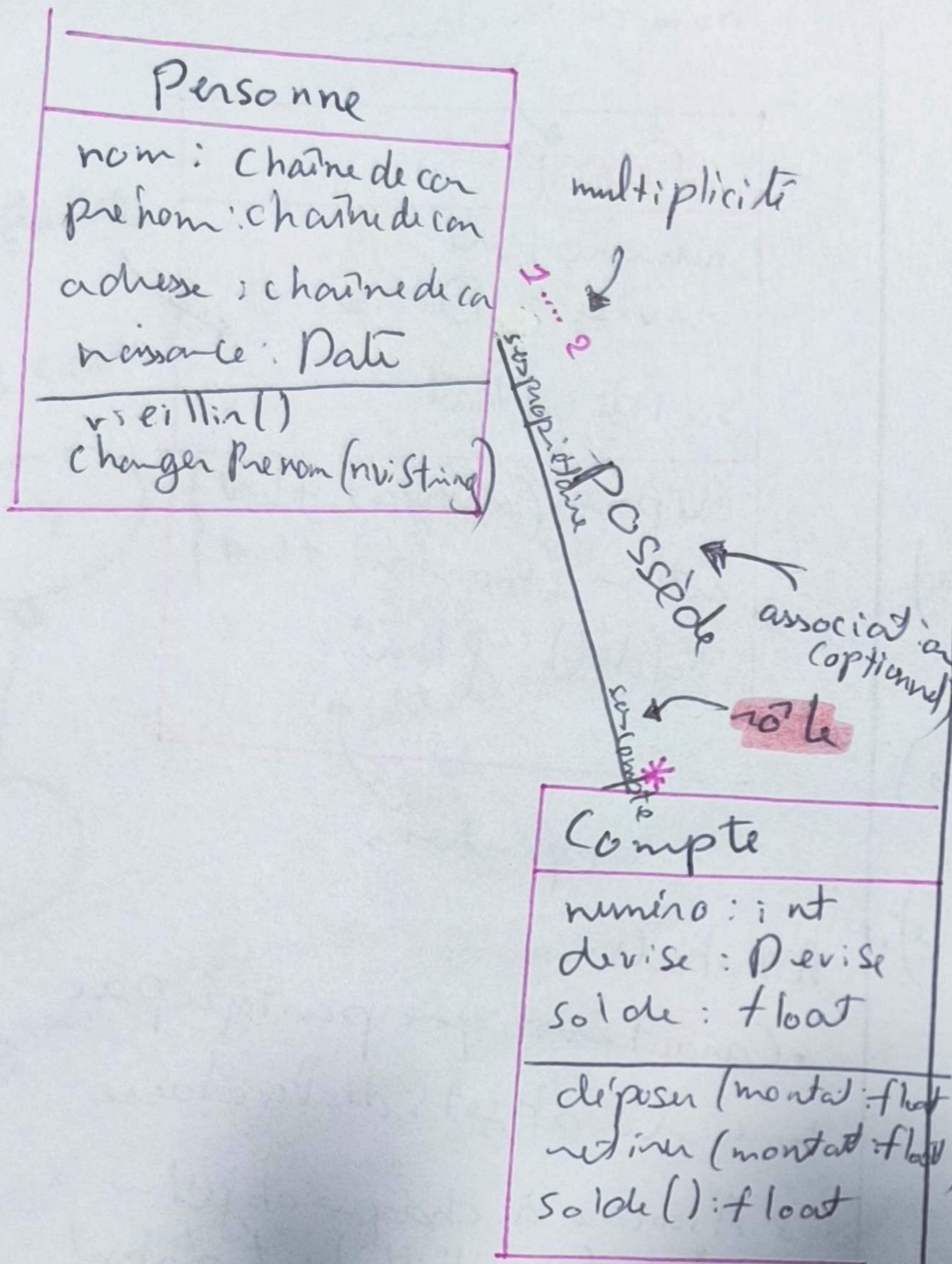
Service qui peut être demandé à tout objet

- Comportement commun

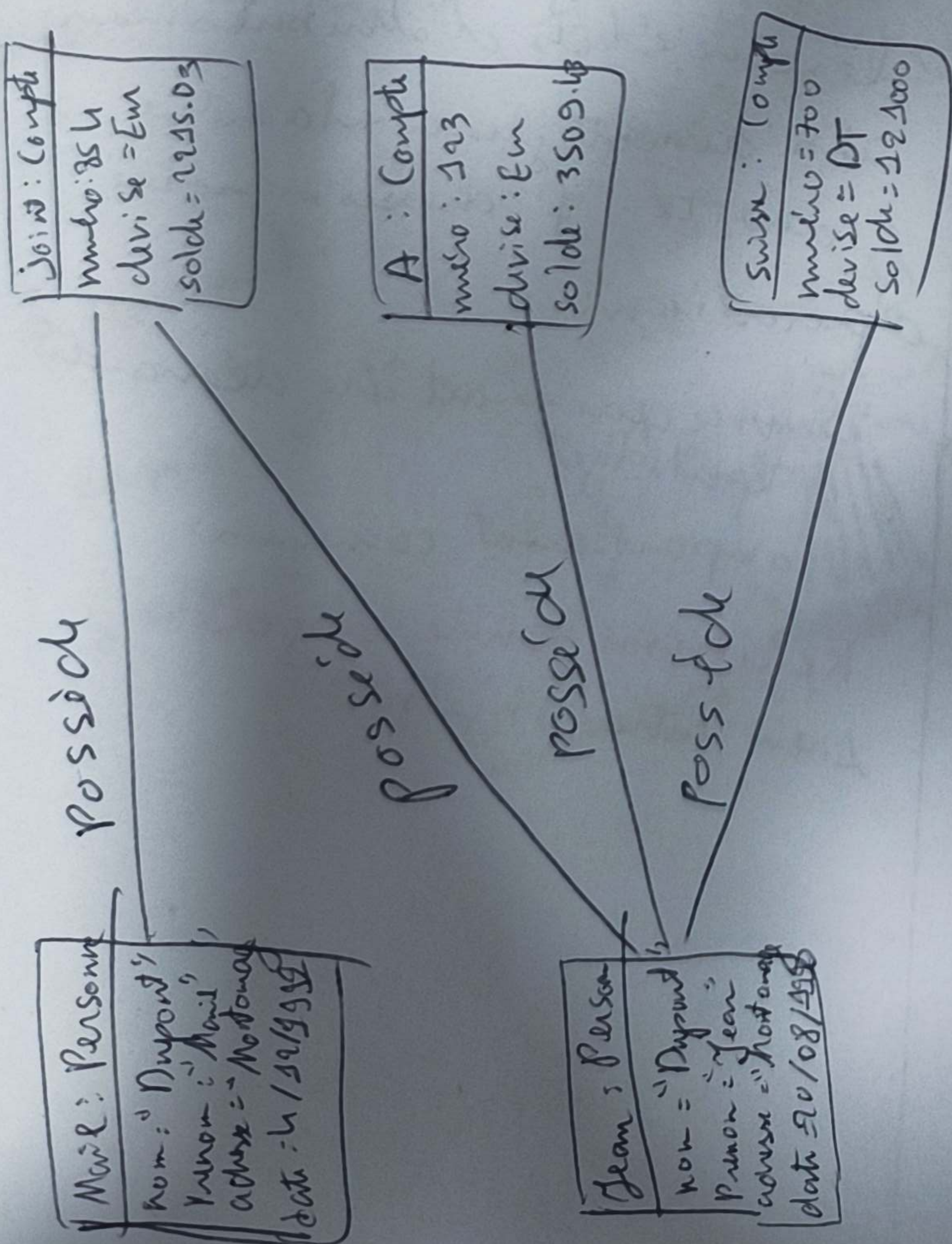
Relation entre objets

Lien entre objets

soit l'association :



donnant par exemple



=> Mr Dupont possède 3 comptes dont 1 de ses comptes est

possédé aussi par Mdm Marie

=> le compte joint possède 2 propriétaires

**Attribut et association:**

les types des attributs doivent être simple, primitif ou énuméré.

Signifie, par d'attribut dont le type est une **classe du diagramme** mais il faut avoir une association vers cette classe du diagramme on peut alors différencier entre 2 type de classe :

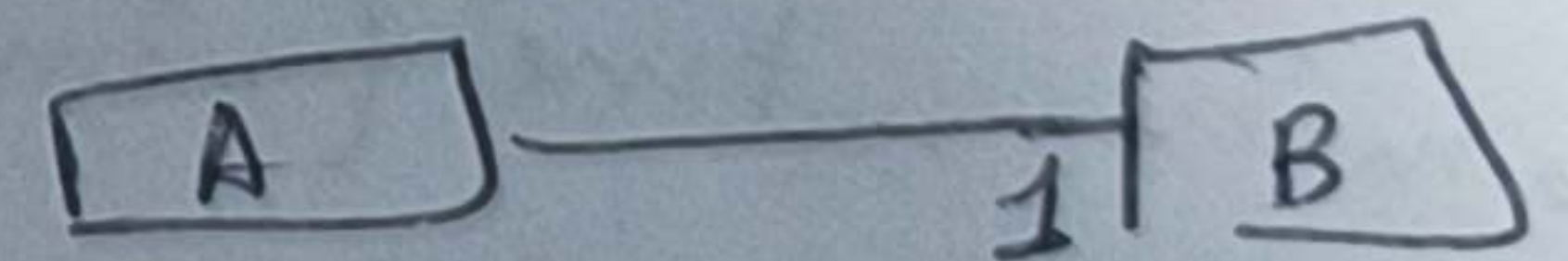
- classe de type donnée standard (devise, date...)

- classe du module LML (Personne)

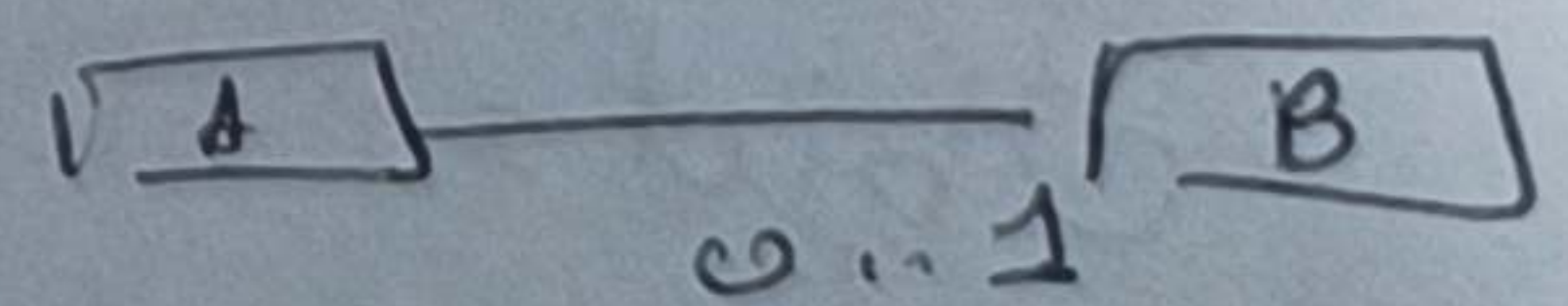
le problème ici est sémantique on choisit quelle classe appartient à quel catégorie.

**Multiplicité:**

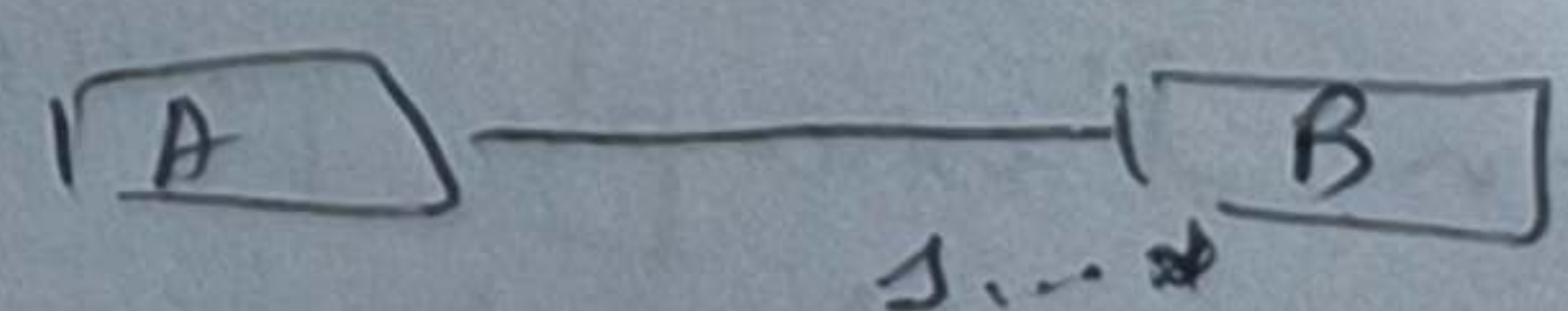
Exactement 1:



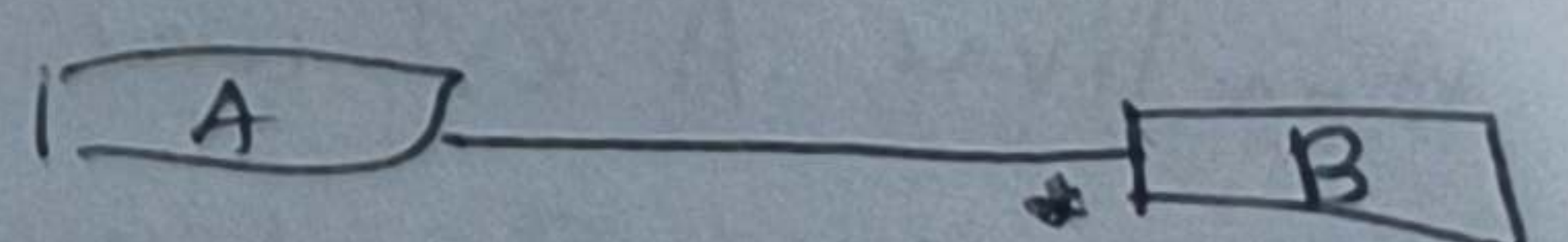
Aut plus 1:



Am moins 1:

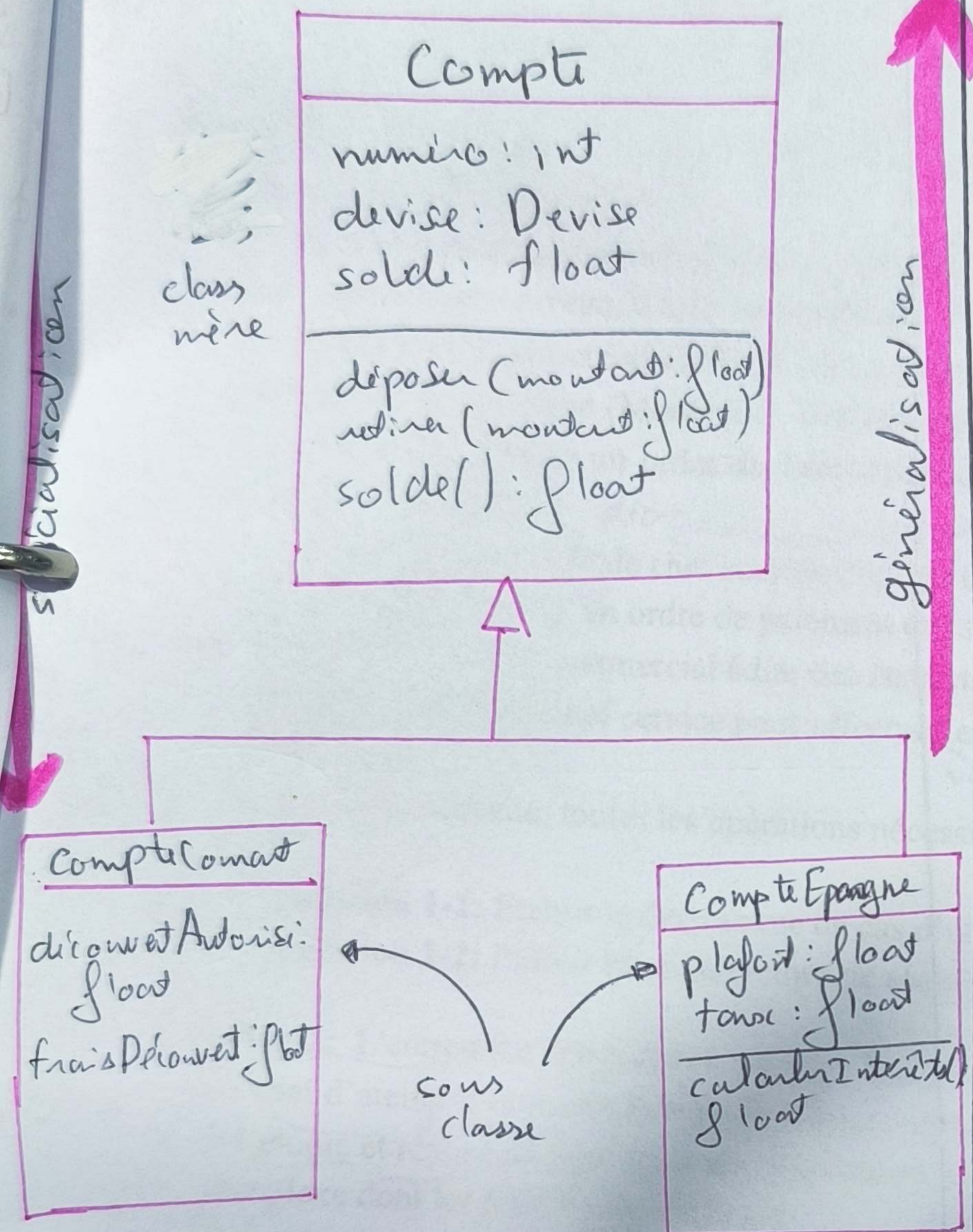


ou plus:



## Archi de classes :

spécialisation / généralisation  
soit le diagramme de classe  
suivant

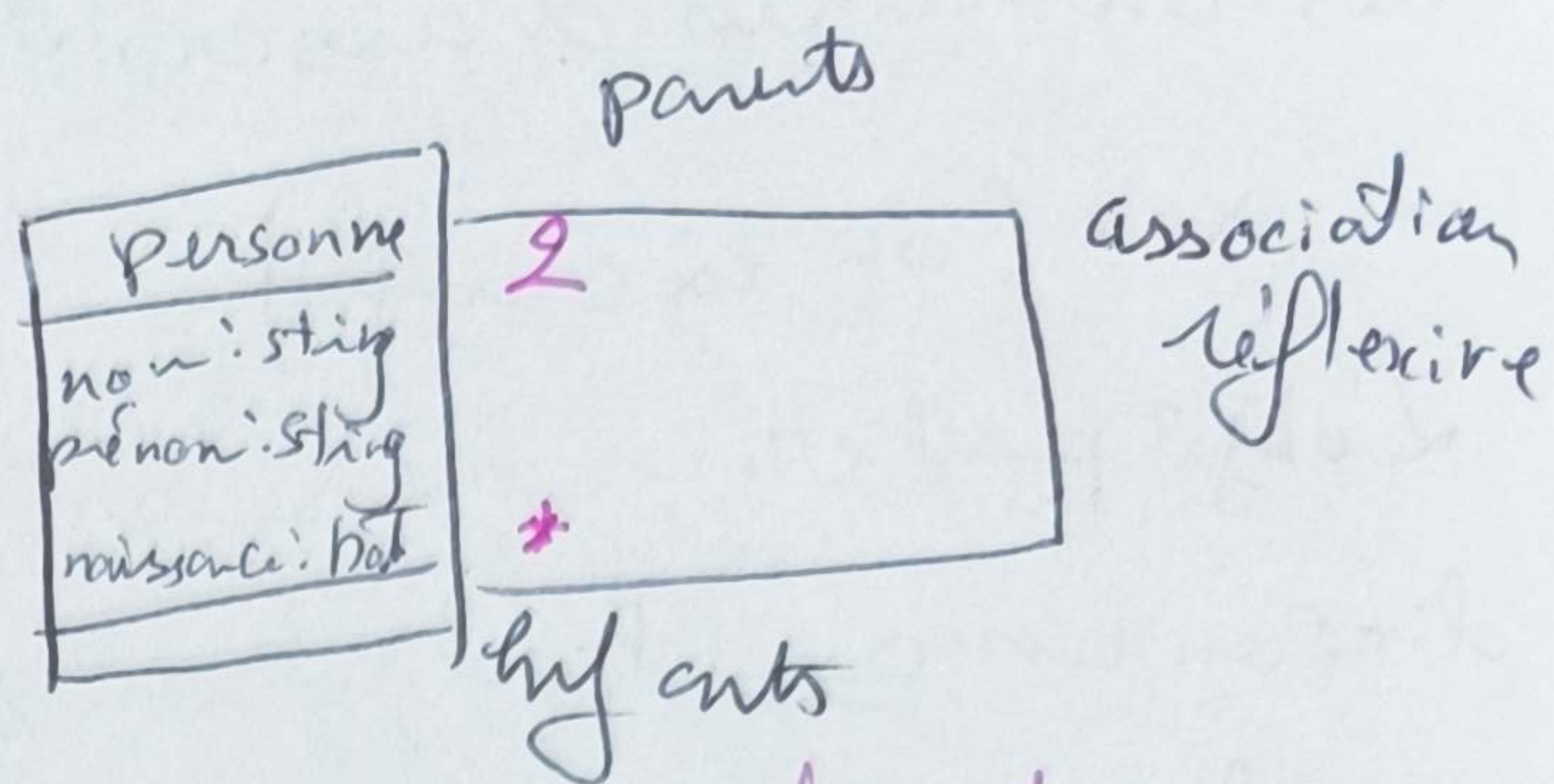


→ les sous classes **CompteComant** et **Compte Epargne** héritent de la classe **Compte**

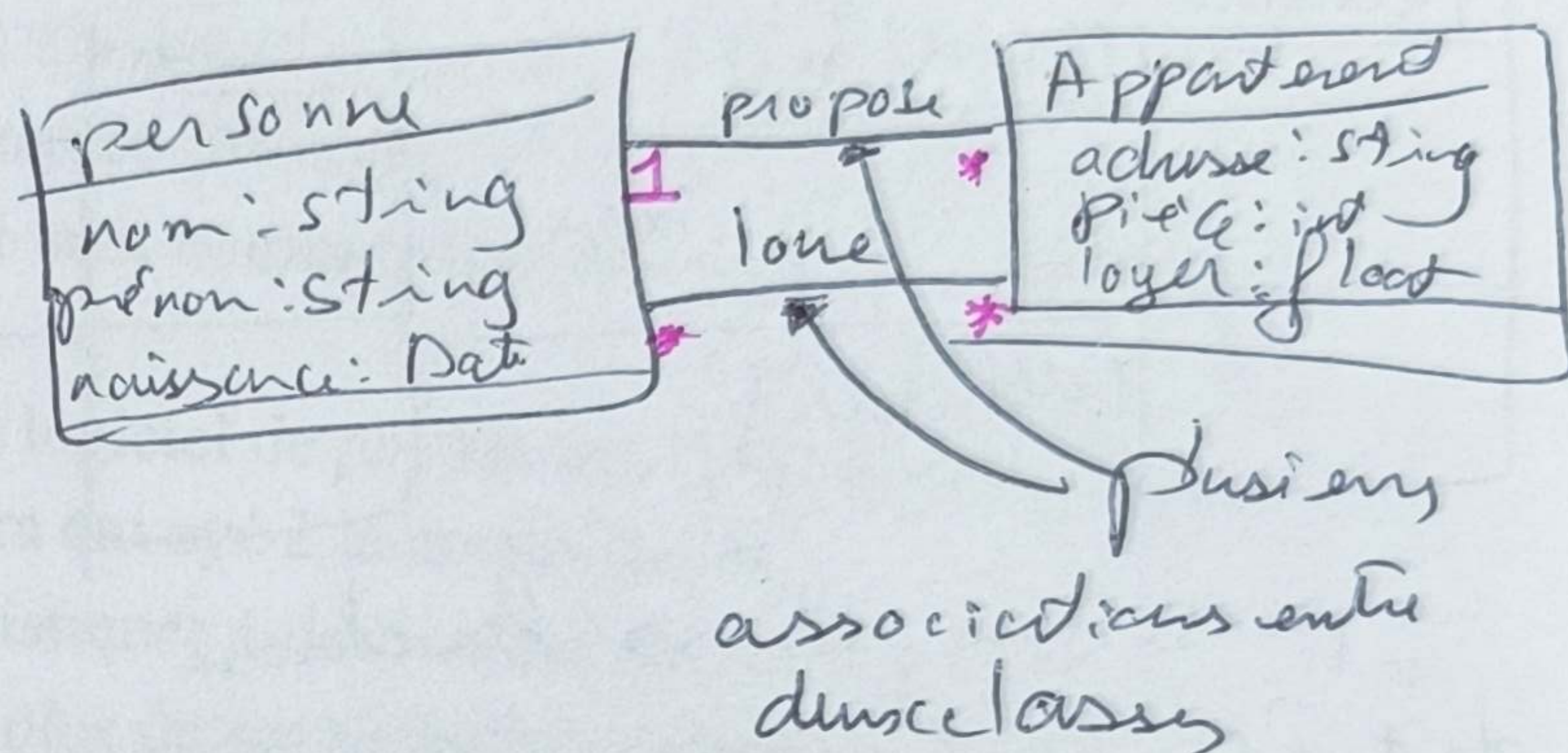
## classe abstraite :

Classe sans instance car elle est une classe mère et elle est inutile dans ce cas comme la classe **Compte** car un compte n'existe pas en soi

## Association réflexive :



**Association multiple :**  
une classe peut avoir plusieurs associations



## Les contraintes :

on peut différencier entre 2 types :

- contraintes de collection :

- **ordered** : les éléments doivent être ordonnés et unique
- **unique** : chaque élément de la collection doit être unique. Une classe ne peut pas avoir plus d'un étudiant
- **bag** : collection non ordonnée et non unique, l'ordre n'a pas d'importance et on a des doublons
- **sequence** : collection ordonnée et les doublons sont autorisés

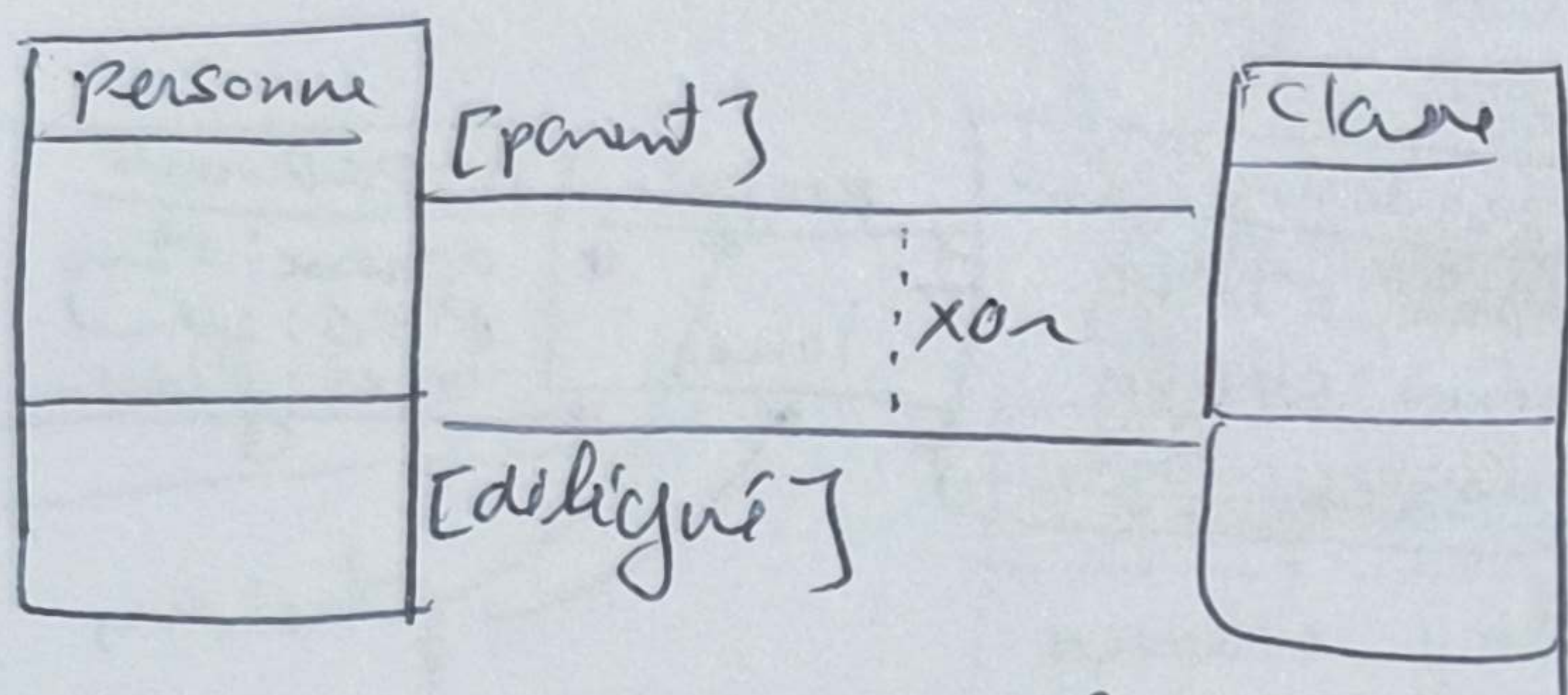
ordered = sequence + unique

normalisation

- des contraintes d'association

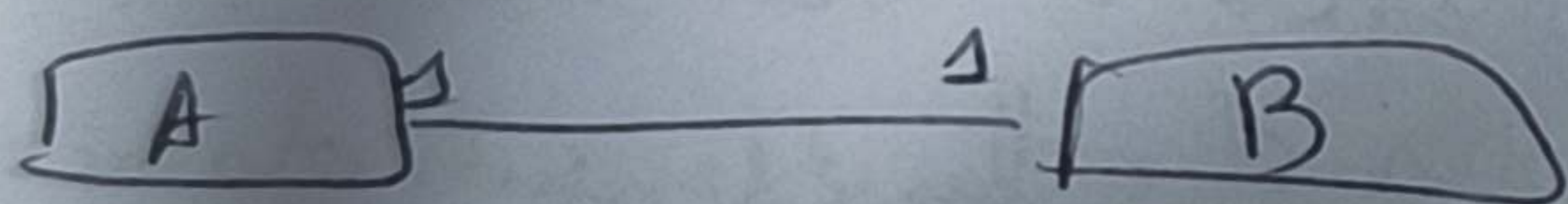
- xor (ou exclusif)

L'objet peut être lié selon  
la son un ou la liaison 2  
par les 2 en même temps comme  
par exemple dans la classe:



une personne dans la classe  
doit être soit un parent soit un délégué  
ne peut pas être les 2 en même temps

- contrainte de multiplicité:  
par exemple



la liaison doit exister